

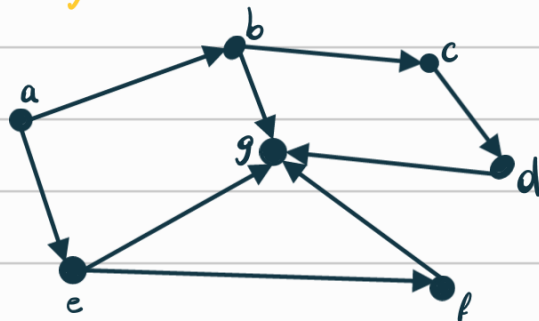
grafo

Nodos conectados entre si.

¿Qué es? : 2 conjuntos; 1 conj^o de vértices (nodos) y conj^o E de arcos.
Arcos representan uniones que pueden tener entre esos vértices.
No todos los vértices se conectan con todos.

Tipos de grafos:

Dirigidos: conexión entre 2 vértices tiene una dirección.



$$V = \{a, b, c, d, e, f, g\}$$

$$E = \{(a,b), (b,c), (c,d), (a,e), (e,f), (e,g), (d,g), (f,g), (b,g)\}$$

~ El arco (u,v) es un par ordenado que indica una flecha del vértice u al vértice v . ~ **Orden es Importancia.**

No Dirigidos: conexión entre 2 vértices No tiene dirección.

Arcos Emergentes: Para un vértice V se denominan a los arcos que salen de V .

Grado de Emergencia: Para un vértice V se denominan a la CANTIDAD de arcos que salen de ese vértice.

Hojas de grafo dirigido: Conjunto de vértices cuyo grado de emergencia es 0 (no salen arcos, solamente llegan).

Arcos Incidentes: Para un vértice V se denominan todos los arcos que llegan a V .

Grado de Incidencia: Para un vértice V se denominan a la cantidad de arcos que llegan

a un vértice.

Raíz De Grafo Dirigido: Vértice al que no llega nadie (grado de incidencia es 0).

Grafo no Dirigido y Grado: No se habla de incidencia o emergencia ya que no hay dirección, $\therefore$ se habla del grado del vértice.

Grado: Cantidad de arcos que inciden en el vértice.

$$\text{Grado}(b) = 3$$

$$\text{Grado}(p) = 4$$

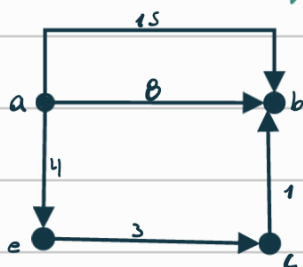


GRAFO PESADO: Los arcos También Tienen información $\rightarrow$ El vínculo entre 2 nodos (vértices) del grafo también va a tener un valor asociado, se denomina Peso.

Si grafo represento mapas con ciudades $\rightarrow$ arcos = caminos.

Peso = distancia.

Multi (di) Grafo: Hay más de un arco entre 2 vértices (arcos paralelos)



Hay 2 arcos que unen "a" con "b", uno con peso 8 y otro con peso 15.

Camino: Secuencia alternante de vértices y arcos, cada arco es incidente en su vértice precedente y sucesor. Empiezo en U y termino en W.

Ciclos:

Simple: Comienzo y termino en el mismo vértice sin repetir arcos.

Dirigido: Aristas Tienen dirección y son recorridas en su dirección.

Aciclico: Grafo sin ciclo.

Ej. Difícil Final: Dado un grafo, verificar si es un camino.

Dado un grafo, verificar si es un árbol.

Dados 2 vértices buscos cuál es el camino MENOS pesado.

COSTO de un CAMINO (ciclo): Suma de los pesos de los arcos que forman el camino (ciclo).

Subgrafo: Subconjunto de vértices y arcos de otro grafo original sin agregado adicional.

Grafo Conexo: Para 2 vértices hay un camino entre ellos, desde cualquier parte puede llegar a cualquier otra parte, sino = **DISCONEXO**.

TDA: Grafo no Dirigido

Graph $\langle V, E \rangle$: Grafo formado de vértices con rótulos de tipo V y arcos con rótulos de tipo E .

VERTEX $\langle V \rangle$: La posición de un vértice con rótulo de tipo V .

EDGE $\langle E \rangle$: La posición de un arco con rótulo de tipo E .

VERTICES $()$: Retorno colección iterable con todos los vértices del grafo.

Edges $()$: Retorno colección iterable con todos los arcos del grafo.

IncidentEdges (v) : Retorno colección iterable con todos los arcos incidentes sobre un vértice v .

opposite (v, e) : Retorno el otro vértice w del arco $e = (v, w)$. Error si e no es incidente (o emergente de v).

endVertices (e) : Retorno un arreglo (2 componentes) conteniendo los vértices del arco e .

areAdjacent (v, w) : Testeo si los vértices v, w son adyacentes.

replace (v, x) : Reemplazo rótulo del vértice v con x .

replace (e, x) : Reemplazo rótulo de arco e con x .

insertVertex (x) : Inserto y retorno un nuevo vértice con rótulo x .

insertEdge (v, w, x) : Inserto un arco con rótulo x entre los vértices v y w .

removeEdge (e) : Elimino el arco e y retorno el rótulo almacenado en e .

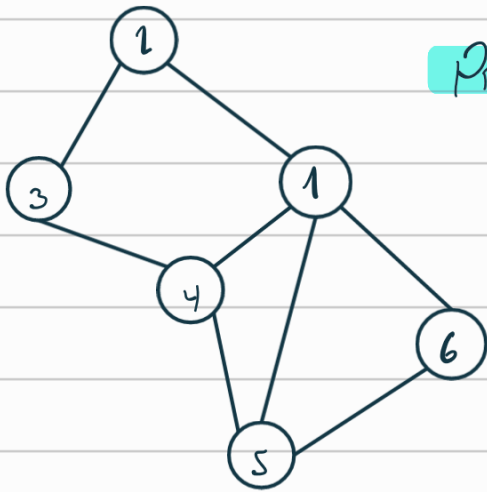
removeVertex (v) : Elimino el vértice v y todos sus arcos adyacentes y retorno el rótulo de v .

Recorridos de grafos

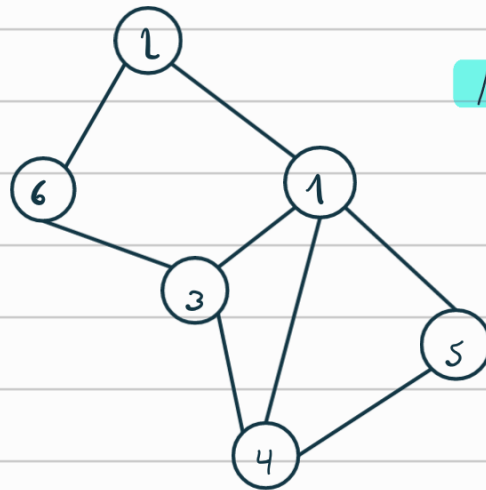
1. PROFUNDIDAD: Equivale a recorrido pre o pos orden en árboles + Testeo para no volver a recorrer un subgrafo ya explorado.

2. ANCHURA: Equivale a recorrido por niveles en árboles + un Testeo para no volver a recorrer un subgrafo ya explorado.

• Debo elegir por donde empezar, si no dice a partir del vértice ..., $\rightarrow$ el primer iterador de vértice y arrancar del 1^{er} vértice que venga en el iterador.



PROFUNDIDAD



ANCHURA

Profundidad:

- Recorre todos los v de manera ordenada.
- No expandiendo todos los nodos que va localizando, de forma recurrente y en un camino concreto.
- Cuando no quedan más nodos que visitar en dicho camino, regreso (Backtracking), repitiendo el mismo proceso con cada hermano del nodo ya procesado.

ALGORITMO 1 DFS (G : grafo; V : vértice)

Procesamiento de V previo al recorrido (Visita Pre)

marcar a V como visitado

Para cada vértice W adyacente a V en G hacer

Si W no está visitado $\rightarrow$ DFS (G, W)

Procesamiento de V posterior al recorrido (Visita Pos)

ALGORITMO 2 DFSShell (G : grafo)

Para cada vértice V de G hacer

marcar V como no visitado

Para cada vértice U de G hacer

Si U no está visitado entonces

DFS (G, U)

Cómo hacer UnionFind

1. Usar mapeo externo al Grafo de Vertex en Boolean

Ventaja: no hay que modificar el grafo que ya programamos.

Contra: El tiempo de marcar y desmarcar puede tender a $O(n)$.

2. Agregar un boolean a la clase Vértice del grafo.

Ventaja: Garantizo que marcar y desmarcar funciona en $O(1)$.

Contra: Por cada algoritmo que escribo escribo el propio agregando atributos y operaciones.

Hay que modificar la `INTERFACE`

3. Decorar los vértices del grafo (opción de GT).

Ventaja: En forma abstracta mantengo toda la información del DFS y de futuros algoritmos.

Contra: Hay que modificar lo que ya programamos.

Recorrido en PROFUNDIDAD: haber si $\exists$ camino entre 2 vértices.

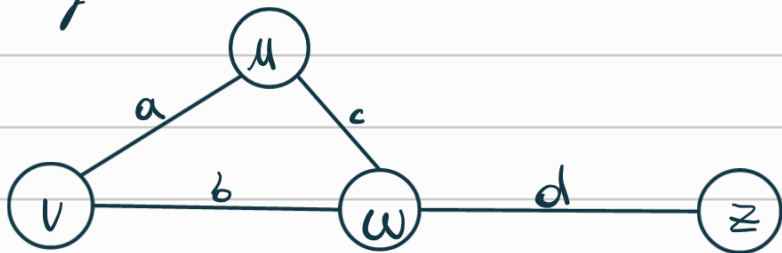
Decorar los Vértices: Una posición decorada es una posición que además es un nodo.

Busqueda en Ancho:

- Algoritmo para recorrer o buscar elementos en un grafo.
 - Comienzo eligiendo algún nodo como elemento raíz y se exploran todos los vecinos de este nodo.
 - Para cada uno de los vecinos se exploran sus respectivos vecinos adyacentes, y así hasta que se recorra todo el grafo.
- 

Estructura para Grafos

Lista de Arcos: Grafo conoce 1 lista de vértices y 1 de arcos (2 listas). Los **vértices** conocen su **etiquetas** respectivos y su **pos** en la lista de vértices. Los **arcos** conocen su **peso**, los **vértices** que **unen** y su **pos** en la lista de arcos.



Lista de Vértices:



Lista de Adyacencia: Grafo conoce 1 lista de vértices y 1 de arcos.

- Cada **vértice** conoce su **etiquetas** y los **arcos** que **inciden** en él.
- Los **arcos** conocen los **vértices** que **unen** y su **peso**.
- Introduce una lista que, **para cada vértice**, indica los **arcos** que **empezan** cuando el grafo es **dirigido** y los que **empezan/inciden** cuando el grafo es **no dirigido**.

~ A cada v que ya tiene su **etiqueta** y ya conoce su **pos** en la lista de vértices, le **agregamos** una lista propia de **arcos** (solo los incidentes sobre él) ~

¿Qué pasa si es Dirigido?: Un arco puede ser incidente de un vértice pero No del otro extremo, en cambio, en un No Dirigido es incidente de los 2.

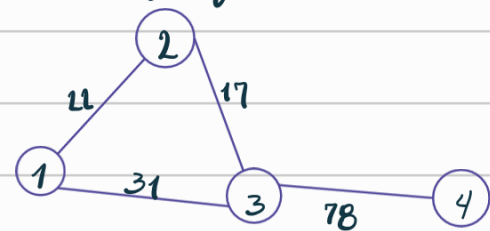
∴ Si mantenemos lista de adyacencia en un **grafo No Dirigido** e inserto un arco, hay 2 listas de adyacencia que **tenemos** que **modificar** porque ese arco es incidente sobre 2 vértices.

En un **Grafo Dirigido** eso No ocurre, solo **modifico** sobre el vértice al cual

está llegando en incidencia al cual está llegando en el arco.

Matriz de Adyacencia: El grafo es un listo de vértices, un listo de arcos y una matriz donde cada componente (i, j) almacena un arco.

- Si los arcos tienen etiquetas y solo puedo tener un arco entre 2 vértices $\rightarrow$ Matriz de Etiquetas.
- Si mi grafo permite múltiples arcos entre 2 vértices $\rightarrow$ Matriz de Listo de Arcos.



	1	2	3	4
1	0	11	31	0
2	11	0	17	0
3	31	17	0	78
4	0	0	78	0

Facilita acceder a la estructura ya que es una matriz (arreglo). $\therefore$ acceso más rápido que si fuera listo de arcos.

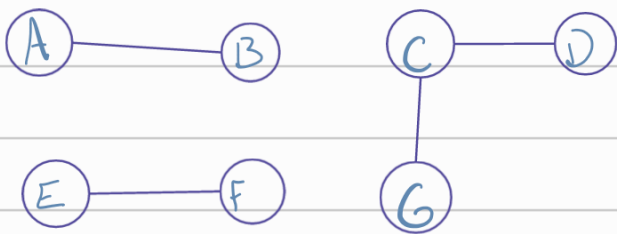
No es tan flexible; eliminación de arcos o vértices puede ser más problemático.

Problemas:

Problema 1: Dado grafo NO dirigido G , retornar mapas con sus componentes conexas numeradas y los vértices que pertenecen a esa componente.

Componentes Conexas:

- Todos los vértices están conectados entre sí (directo o indirectamente).
- No hay conexión con vértices fuera de ese subconjunto.



Componentes conexas:

1. $\{A, B\}$
2. $\{C, D, G\}$
3. $\{E, F\}$

Public Map<Integer, PositionList<Vertex<V>>> componentesConexas(Graph<V, E> g)

Map<Integer, PositionList<Vertex<V>>> nuevo = new TDAMapeo<>();

Map<Vertex<V>, Boolean> visitados = new TDAMapeo<>();

For (Vertex<V> v: g.vertices())

visitados.put(v, false);

int i = 1;

For (Vertex<V> v: g.vertices()) {

IF (!visitados.get(v)) {

PositionList<Vertex<V>> l = new TDALista<>();

DFS(g, v, visitados, l);

nuevo.put(i, l);

i++;

}

return nuevo;

private void DFS(Graph<V, E> g, Vertex<V> v, Map<Vertex<V>, Boolean> visitados, PositionList<Vertex<V>> l) {

visitados.put(v, true);

l.addLast(v);

For (Edge<E> e: g.incidentEdges(v)) {

Vertex<V> ady = g.opposite(v, e);

IF (!visitados.get(ady))

DFS(g, ady, visitados, l);

Problema 2: Dado GND y 2 vertices, retorne camino entre ambos vertices.
1: $A \rightarrow$ retorne nulo.

```
public PositionList<Vertex<V>> camino (Vertex<V> v1, Vertex<V> v2, Graph<V,E> g)
{
    PositionList<Vertex<V>> l = new TDAList<>();
    Map<Vertex<V>, Boolean> m = new TDAMap<>();
    For (Vertex<V> u: g.vertices())
        m.put (u, false);
    boolean hay = DFS (g, v1, v2, l, m);
    IF (hay)
        return l;
    else
        return null;
}
```

```
private boolean DFS (Graph<V,E> g, Vertex<V> v1, v2, Pl<Vertex<V>> camino, Map<Vertex<V>, Boolean> m)
{
    camino.put (v1, true);
    v1.visitados.addLast(v1);
    Position<Vertex<V>> pos = camino.last() - Quando por favor BackTracking
    IF (v1 == v2)
        return true;
    Else {
        boolean encuentre = false;
        For (Edge<E> e: g.incidentEdges (v1))
        {
            Vertex<V> nuevoOrigen = g.opposite (origen, e);
            IF (!visitados.get (nuevoOrigen))
            {
                encuentre = DFS (g, nuevoOrigen, v2, camino, visitados);
                IF (encuentre)
                    return true;
            }
        }
    }
}
```


// BACKTRACKING (no encontré camino por esa rama → saca vértice del camino
// y vuelvo a entrar.

```
Camino.remove(pos);  
return false;  
}
```

TAREA: Implementar método para hallar ciclos en Java.

Ciclo Simple:

- Camino que empiezo y termino en mismo vértice.
- No repite aristas (arces).
- Tiene al menos 3 vértices (en GND). → GUARDO el padre.

Algoritmo:

1. Empiezo de un vértice origen.
2. Exploro vecinos No visitados.
3. Si encuentro vecino que no es origen y No por donde vine → Ciclo.
4. BACKTRACKING si no encuentro ciclo por esa rama.

```
public PosicionList<VertexUV> hallarCiclo(Graph<UV, E> g, VertexUV origen) {  
    PosicionList<VertexUV> toRet = new TDALista<>();  
    Map<VertexUV, Boolean> visitados = new TDAMap<>();  
    try {  
        for (VertexUV u : g.vertices())  
            visitados.put(u, false);  
        { catch (InvalidKeyException e) {  
            e.printStackTrace();  
        }  
        boolean hay = DFSBuscarCiclo(g, origen, null, origen, toRet, visitados);  
        IF (hay)  
            return toRet;  
    }  
}
```

```

else
    return null;
}

private boolean DFSBuscarCiclo (G<V,E> g, V<V> actual, Padre, origen, PL<V<V>> ciclo, M<V<V>> visio)
{
    visitados.put(actual, true);
    ciclo.addLast(actual);
    Position<Vertex<V>> posEnCiclo = ciclo.last() // Guardo pos para BACKTRACKING.
    for (Edge<E> e: g.incidentEdge(actual)) {
        Vertex<V> vecino = g.opposite(actual, e);
        IF (!visitados.get(vecino)) {
            boolean encuentre = DFSBuscarCiclo(g, vecino, actual, origen, ciclo, visitados);
            IF (encuentre)
                return true;
        }
        else { // Vecino está visitado. NO volví por donde vine
            IF (vecino == origen && vecino != padre) {
                ciclo.addLast(origen);
                return true;
            }
        }
    }

    // BACKTRACKING (No encuentre ciclo por esa rama → deshago camino)
    ciclo.remove(posEnCiclo);
    return false;
}

catch (IKE | IVE | IEE | ELE | IPE e) {
    e.printStackTrace();
}

return false;
}

```

Ejercicio 3: Dado grafo con V retulados con caracteres y arcos retulados con números enteros y 2 vértices origen y destino, método que halle camino con costo mínimo.

Objetivo: Encontrar camino c costo $\rightarrow$ comparar costos y me quedo con c .

Cuando Para: Exploro todos los caminos posibles.

Visitados: marco, exploro, Desmarco (Backtracking completo). $\rightarrow$ exploro otros caminos.

BFS (Búsqueda en Anchura): Garantizo encontrar camino más corto en número de pasos/aristas.

¿Cómo Funciona? Exploro grafo por niveles.

Nivel 0: origen.

Nivel 1: todos los vecinos del origen.

Nivel 2: todos los vecinos de los vecinos.

... Así sucesivamente

$\rightarrow$ Garantizo encontrar camino más corto porque exploro 1º los vértices más cercanos.

Estructura Clave: COLA

- Uso cola en vez de recursión.
- Encojo vecinos a medida que los descubro.
- Desencojo para procesarlos en orden.

Algoritmo:

1. Creo una cola y 1 mapa (Vértice $\rightarrow$ Distancia).
2. Encojo el origen con Distancia = 0.
3. Mientras la cola No esté vacía:
 - Desencojo 1 vértice.
 - Si es el destino $\rightarrow$ retorno su distancia.
 - Para cada vecino No visitado:
 - * Le asigno: $\text{distancia} = \text{distancia_actual} + 1$

- lo marco como visitado (poniéndolo en el mapa).

- lo encolo.

4. Si Termino sin encontrar el destino $\rightarrow$ no hay camino (retorno -1).

$O(n + m)$ donde $n = \text{vértices}$, $m = \text{aristas}$.

- Retorna la distancia o longitud del camino mas corto.

¿Siempre que me pasen Vertice por parametro hago CheckVertex?

¿BFS Sirve para camino de costo mínimo?

No, BFS solo sirve para:

- grafos sin pesos (o con pesos iguales)
- Encontrar el camino con menos aristas.

PARA GRAFOS PESADOS necesito:

- DFS con Backtracking, exploro todos los caminos.
- DIJKSTRA.

